DiT-AR v5: A Sliding-Window Conditional Diffusion Transformer for Generative GPU Power-Trace Synthesis on HPC Systems

Yuzhang Lin
Assistant Professor
New York University
yl12252@nyu.edu

Abhishek Agrawal
Graduate Student
New York University
aa9360@nyu.edu

Haoyang Li
PhD Student
New York University
hl4814@nyu.edu

***Abstract*—Time-resolved per-job GPU power consumption is a key quantity for energy-aware scheduling, thermal design, and carbon accounting in modern AI/HPC clusters, yet it is rarely available in the design loop because high-resolution telemetry is operationally sensitive and expensive to release. We address this gap with a generative pipeline that jointly ingests SLURM scheduler metadata and sub-second multi-channel GPU telemetry from MIT SuperCloud, and learns to synthesize statistically realistic GPU power trajectories at $\Delta t \approx 103$ ms resolution. Our model, DiT-AR v5, is a 33.7-million-parameter diffusion transformer that operates on three aligned patch-token streams (long multivariate context, future auxiliary non-power channels, and the noisy power target) plus a 24-dimensional SLURM conditioning prefix token. Training uses v-prediction on a cosine noise schedule with classifier-free guidance dropout, an EMA shadow, and scheduled sampling that fades from teacher-forced to autoregressively-fed context. Inference is sliding-window autoregressive with non-overlapping strides, DDIM-50, and CFG $= 3.0$. On a job-disjoint test set of 4,404 held-out HPC jobs ($\approx$ 113 days of GPU wall-time), DiT-AR v5 attains a median Pearson $r = 0.109$, mean $r = 0.212$, with 32.4% of jobs above $r = 0.30$ and 18.0% above $r = 0.50$; median per-job RMSE is 20.9 W and median MAE 15.0 W. We provide a full reproducibility artifact (ETL, configs, checkpoints, evaluation scripts) and analyze failure modes that follow directly from the heavy-tailed nature of HPC GPU power.**

***Index Terms*—GPU power modeling, HPC energy, SLURM, diffusion transformer, classifier-free guidance, time-series generation, autoregressive inference, MIT SuperCloud, generative AI for systems.**

The full artifact (ETL, configs, training and inference scripts) is released here.

I. Introduction

THE carbon and energy footprint of GPU-accelerated computing has become a first-order operational concern for university clusters, hyperscale data centers, and national HPC facilities alike [1], [2]. Time-resolved per-job GPU power consumption is the elementary observable that downstream tools–energy-aware schedulers, cooling controllers, capacity planners, and carbon-accounting dashboards–need as input. However, sub-second telemetry traces joined to the scheduler are rarely shareable in practice: they are operationally sensitive, vary across hardware generations, and are expensive to store and curate. **This work asks whether a single conditional generative model can synthesize realistic high-resolution GPU power traces from scheduler metadata alone, faithfully enough to be useful in the design loop for the systems that consume them.**

We target the MIT SuperCloud telemetry export, in which each job exposes per-second GPU statistics aligned with a SLURM accounting record [3]. After cleaning, each job becomes a four-channel time series at $\Delta t \approx 103$ ms: GPU utilization, GPU memory utilization, allocated memory in MiB, and the regression target, instantaneous power draw in watts. We learn $p(\,\mathrm{power}_{[t,t+W]} \mid \mathbf{X}_{\mathrm{ctx}}, \mathbf{A}_{\mathrm{aux}}, \mathbf{c}_{\mathrm{slurm}}\,)$ where the SLURM context $\mathbf{c}_{\mathrm{slurm}} \in \mathbb{R}^{24}$ is a fixed-length vector of cyclical, scalar, and one-hot features, and $\mathbf{A}_{\mathrm{aux}}$ supplies the non-power telemetry over the prediction window. The auxiliary signals are treated as *observed* at inference; we discuss this oracle assumption explicitly in Sections IX and IX-A, and why it is the right scaffolding for both near-term "what-if" workflows and longer-term end-to-end forecasting.

Contributions. We make four contributions:

1) A reproducible three-stage ETL that turns raw GPU CSVs and SLURM logs into job-level, train/test-disjoint, memory-mapped NumPy traces at $\sim$103 ms resolution, with train-only normalization statistics and atomic per-job writes (§III).
2) **DiT-AR v5**, a conditional diffusion transformer with three aligned 1-D patch streams, a SLURM prefix token, AdaLN-Zero modulation from joint timestep+SLURM embeddings, v-prediction on a cosine schedule, classifier-free guidance, EMA, and scheduled sampling (§IV–§V).
3) A sliding-window autoregressive inference procedure (DDIM-50, CFG=3, non-overlapping stride) that stitches per-window denoised power into long multi-hour trajectories without retraining (§VI).
4) An empirical study on 4,404 held-out HPC jobs covering $\approx$113 days of GPU wall-time, with robust statistics, distributional plots, qualitative trace overlays, and a structured failure-mode analysis (§VIII–§IX).

The remainder of the paper is organized as follows. Section II reviews diffusion models and HPC power modeling. Sections III–IV describe the data pipeline and the DiT-AR v5 architecture. Section V details training and Section VI the autoregressive inference loop. Section VII states the experimental protocol; Section VIII reports quantitative and qualitative results. Section IX discusses limitations and Section X concludes.

II. Background and Related Work

HPC power and energy modeling. Classical approaches estimate aggregate cluster power from running average power limit (RAPL) counters, vendor telemetry, and node-level utilization regressions [1], [4]. Recent work pairs queue-level features with aggregate power forecasts [5], [6], but public artifacts that release *job-granular, sub-second* GPU traces aligned with the scheduler–and tackle the inverse problem of *generating* them from metadata–are scarce. The MIT SuperCloud workload dataset [3] makes this study possible by exposing both the SLURM event stream and per-job GPU CSVs at $\sim$100 ms resolution.

Diffusion models for continuous data. Denoising diffusion probabilistic models [7], [8] sample by reversing a forward Gaussian noising process, trained with a noise- or signal-prediction objective. The v-prediction parameterization of Salimans and Ho [9] stabilizes training on continuous targets across signal-to-noise regimes. **Classifier-free guidance** [10] jointly trains a conditional and unconditional model under a Bernoulli mask and trades sample diversity for adherence to conditioning at inference, a property we exploit because SLURM metadata is the operationally relevant prior. **DDIM** [11] provides a deterministic ($\eta = 0$) accelerated sampler that we use to keep autoregressive inference tractable at long horizons.

Diffusion transformers. Diffusion transformers (DiT) [12] replace the U-Net backbone with a ViT-style patch-and-transformer architecture conditioned via adaptive layer normalization (AdaLN-Zero) driven by a global embedding. This makes scaling and attention-mediated long-range modeling natural, which is the desideratum for HPC power because job-internal phases (initialization, compute, I/O, checkpointing) span thousands of bins.

Time-series diffusion. Closest to our setting are TimeGrad [14], CSDI [15], and diffusion-style probabilistic forecasters built on attention backbones. These are typically trained on short horizons or impute missing values; we adapt the DiT formulation to (i) conditional power-only synthesis with multivariate context and (ii) sliding-window autoregressive deployment over hours of telemetry.

III. Data and Preprocessing

A. Raw inputs and selection

We use the public MIT SuperCloud workload export (2022Q1) [3], which provides per-job GPU CSVs and a global SLURM accounting log. The repository ships the cleaning pipeline but does not redistribute the raw data. After an inner join on `job_id`, all jobs with at least one valid GPU CSV are retained. A job-level disjoint test split is taken with `test_frac` $= 0.10$ and `seed` $= 42$.

TABLE I
Channel layout of each per-job NPY array.

Idx	Channel	Normalization	Role
0	`gpu_used_pct`	linear $(x/100)\cdot 2-1$	ctx + aux
1	`memory_used_pct`	linear $(x/100)\cdot 2-1$	ctx + aux
2	`memory_used_MiB`	$\log 1p$ + train z-score	ctx + aux
3	`power_draw_W`	$\log 1p$ + train z-score	ctx + **target**

TABLE II
SLURM conditioning vector $\mathbf{c} \in \mathbb{R}^{24}$.

Group	Dim.	Features
Calendar harm.	4	`hour_{sin,cos}, dow_{sin,cos}`
Resource scale	7	`mem_unlimited, mem_req_scaled, nodes_req_is_one, nodes_req_log, cpus_req_scaled, duration_scaled, priority_scaled`
Priority tier	3	one-hot `ptier_{0,1,2}`
Job type	4	one-hot `type_{batch, interactive, map, other}`
State bucket	6	one-hot `state_{other, 3, 4, 5, 6, 11}`
Total	**24**	

B. Three-stage ETL

Stage 1 – GPU ETL. For each job, GPU CSVs are discovered, validated against the expected sampling-interval range $[\delta_t^{\min}, \delta_t^{\max}]$, binned to a dense timeline, and aggregated across nodes (mean or sum per metric, configured per channel). A `min_node_keep_ratio` threshold drops jobs where too few nodes report. Output is one atomic Parquet per job with footer metadata (`job_id, length, duration_sec, delta_t, nodes_used`) and an index file for downstream joins.

Stage 2 – SLURM ETL. The SLURM table is inner-merged with the GPU index on `job_id`. We engineer 24 conditioning features (Table II): calendar harmonics from submit time, quantile-rescaled scalar requests, and one-hot encodings for priority tier, job type, and final state bucket.

Stage 3 – SeqETL. Surviving joined jobs are materialized as $(4, T)$ `float32` NPY arrays following the channel layout in Table I. Train-only $\log(1+x)$ + z-score statistics for memory and power are saved in `norm_stats.npz` and applied to both splits, eliminating leakage. A chunk index lists every (job, window-start) tuple satisfying length and padding constraints. The runtime dataset (`PowerTraceDataset`) memory-maps the per-job NPYs with an LRU cap on open file handles, applies zero-padding for partial windows at job boundaries, and emits a `pred_mask` marking valid bins; all loss and metric computations are masked.

C. Window geometry

The deployed configuration uses $W_{\text{ctx}} = 16{,}384$ context bins, $W_{\text{pred}} = 8{,}192$ prediction bins, patch size $P = 16$, and a non-overlapping stride $= W_{\text{pred}}$. At $\Delta t \approx 103\,\text{ms}$ this is roughly 28 min of context and a 14 min prediction horizon per autoregressive step.

D. Dataset scale

After cleaning, the train split contains **39,632 jobs** producing **124,629 chunks**; the test split contains **4,404 jobs** producing **13,884 chunks**. Total evaluation wall-time is approximately **2,713 GPU-hours** (≈ 113 days), making the held-out set substantially larger than typical benchmark suites in HPC time-series modeling.

IV. DiT-AR v5 Architecture

Fig. 1 summarizes the full forward pass. The five inputs (diffusion timestep, SLURM condition, multivariate context, future auxiliary, and noisy power) are mapped to a unified token sequence of length 2049 through three Conv1d patch embedders and one MLP-projected condition prefix token. A summed time + condition vector drives **AdaLN-Zero** modulation in every one of the 12 DiT blocks. After bidirectional self-attention the last 512 tokens are sliced and reshaped to a v-prediction of width $W_{\text{pred}} = 8192$. The remainder of this section formalises each of these stages.

A. Per-window problem statement

For a given training chunk we observe past 4-channel telemetry $\mathbf{X}_{\text{ctx}} \in \mathbb{R}^{4\times W_{\text{ctx}}}$, future non-power channels 0–2 (the "oracle" auxiliary) $\mathbf{A}_{\text{aux}} \in \mathbb{R}^{3\times W_{\text{pred}}}$, the future normalized power target $\mathbf{p}_0 \in \mathbb{R}^{1\times W_{\text{pred}}}$, the SLURM vector $\mathbf{c} \in \mathbb{R}^{24}$, and a diffusion timestep $t \in \{0, \dots, T-1\}$ with $T = 1000$. The forward process produces $\mathbf{p}_t = \sqrt{\bar\alpha_t}\,\mathbf{p}_0 + \sqrt{1-\bar\alpha_t}\,\boldsymbol{\epsilon}$ with cosine β_t [13]; the network predicts the v-target $\mathbf{v} = \sqrt{\bar\alpha_t}\,\boldsymbol{\epsilon} - \sqrt{1-\bar\alpha_t}\,\mathbf{p}_0$ [9] from $(\mathbf{X}_{\text{ctx}}, \mathbf{A}_{\text{aux}}, \mathbf{p}_t, \mathbf{c}, t)$.

B. Tokenization and prefix

Three independent 1-D patch embedders Conv1d (kernel $=$ stride $= P$, no padding) turn each input stream into a token sequence in $\mathbb{R}^{B\times N\times d}$:

$$\begin{aligned} \mathbf{X}_{\text{ctx}} &\to \mathbf{T}_{\text{ctx}}, \quad N_{\text{ctx}} = W_{\text{ctx}}/P = 1024, \\ \mathbf{A}_{\text{aux}} &\to \mathbf{T}_{\text{aux}}, \quad N_{\text{pred}} = W_{\text{pred}}/P = 512, \\ \mathbf{p}_t &\to \mathbf{T}_{\text{pred}}. \end{aligned}$$

Independent learned positional biases `pos_ctx`, `pos_aux`, `pos_pred` are added; the SLURM prefix has no positional bias. A linear projection `cond_token_proj`$(\mathbf{c})$ yields a single prefix token $\mathbf{t}_0 \in \mathbb{R}^{B\times 1\times d}$. The full sequence is

$$\mathbf{T} = [\,\mathbf{t}_0 \,\|\, \mathbf{T}_{\text{ctx}} \,\|\, \mathbf{T}_{\text{aux}} \,\|\, \mathbf{T}_{\text{pred}}\,],$$

$$N_{\text{tokens}} = 1 + N_{\text{ctx}} + 2N_{\text{pred}} = 2049.$$

C. Conditioning and DiT blocks

A timestep MLP $\psi(t)$ and a SLURM MLP $\phi(\mathbf{c})$ combine additively into a global conditioning vector

$$\mathbf{g} = \psi(t) + \phi(\mathbf{c}) \in \mathbb{R}^{B\times d},$$

which drives **AdaLN-Zero** [12] modulation in every DiT block. Each block is pre-norm with multi-head self-attention and a feed-forward MLP, both branches gated by zero-initialized affine layers conditioned on $\mathbf{g}$. **Classifier-free guidance** replaces $\mathbf{c}$ with a learned null embedding under a Bernoulli dropout mask of probability $p_{\text{cfg}} = 0.05$ during training [10]. A final AdaLN-modulated linear head reads only the last N_{pred} tokens and reshapes their patch logits back to $\hat{\mathbf{v}} \in \mathbb{R}^{B\times 1\times W_{\text{pred}}}$.

D. Capacity and parameter count

The deployed checkpoint uses $d_{\text{model}} = 384$, $n_{\text{layers}} = 12$, $n_{\text{heads}} = 6$, MLP ratio 4, and dropout 0.1, yielding **33.7 M trainable parameters**. Sequence length $N_{\text{tokens}} = 2049$ keeps a single forward pass within H200-class memory at batch size 128 per device, with $\approx$125 GB peak VRAM use under mixed-precision (bf16) training.

V. Training

A. Objective

Let $m \in \{0,1\}^{1\times W_{\text{pred}}}$ be the prediction mask (zeros past job end). Training minimizes a masked v-MSE loss,

$$\mathcal{L} = \frac{1}{\sum m}\,\big\|m \odot (\hat{\mathbf{v}} - \mathbf{v})\big\|_2^2.$$

B. Optimization, regularization, distributed training

Hyperparameters (Table III) are taken verbatim from the shipped `configs/v5.yaml`. We use AdamW with weight decay 0.05, gradient clipping at 1.0, a 500-step linear warmup, and a cosine decay schedule. An EMA shadow with decay 0.9999 tracks weights; the EMA checkpoint is used for all reported inference. Mixed precision (bf16) is enabled; `torch.compile` is intentionally disabled to avoid long shared-cluster kernel compile stalls. The runtime auto-detects available CPUs and GPUs (here, 2×H200 with 128 CPUs) and launches **DDP** with a forkserver `DataLoader` context, an fd-limit raise to 65,536, a CUDA preflight, and graceful checkpoint handlers on `SIGTERM`/`SIGUSR1`.

C. Scheduled sampling

Training is teacher-forced: the context $\mathbf{X}_{\text{ctx}}$ contains *ground-truth* prior power. To narrow the train/inference gap, we activate **scheduled sampling** after 40% of training: with probability p rising linearly from 0 to 0.4, the chunk's context is overwritten with a single-step self-denoised draw. Figure 2 shows that the loss continues to fall after this transition, indicating the model recovers from the distribution shift rather than collapsing.

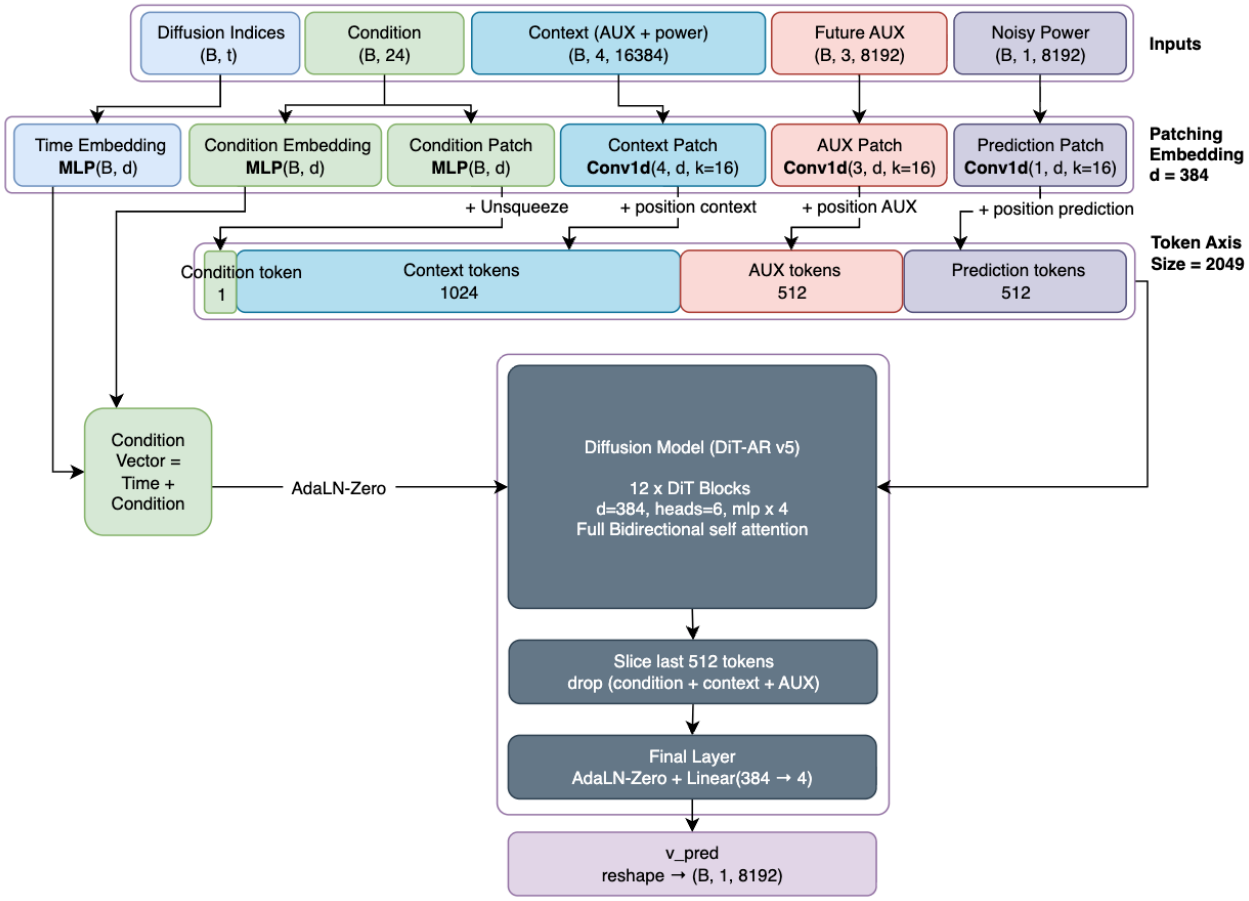


Fig. 1. **DiT-AR v5 forward topology.** Five inputs–diffusion timestep t, 24-D SLURM condition, 4-channel context $\mathbf{X}_{\text{ctx}} \in \mathbb{R}^{4\times 16384}$, future auxiliary $\mathbf{A}_{\text{aux}} \in \mathbb{R}^{3\times 8192}$, and noisy power $\mathbf{p}_t \in \mathbb{R}^{1\times 8192}$–are projected to a common width $d = 384$ via an MLP for the condition prefix and three Conv1d patch embedders (kernel = stride = 16) for context, auxiliary, and noisy-power streams. Independent learned positional biases are added to each patch stream; the prefix has none. The concatenated 2049-token sequence is processed by 12 AdaLN-Zero DiT blocks with full bidirectional self-attention, modulated by the joint condition vector $\mathbf{g} = \psi(t) + \phi(\mathbf{c})$. Only the last $N_{\text{pred}} = 512$ output tokens drive a final AdaLN-Zero linear head whose patch logits are reshaped to the v-prediction $\hat{\mathbf{v}} \in \mathbb{R}^{B\times 1\times 8192}$.

TABLE III
KEY DIT-AR V5 HYPERPARAMETERS (CONFIGS/V5.YAML).

Item	Value	Item	Value
d_{model}	384	n_{layers}	12
n_{heads}	6	MLP ratio	4
Dropout	0.1	Patch P	16
W_{ctx}	16,384	W_{pred}	8,192
Batch (global)	256	Epochs	160
Optimizer	AdamW	Weight decay	0.05
LR (peak)	1.5×10^{-5}	Warmup steps	500
LR schedule	cosine	Grad clip	1.0
CFG drop p	0.05	EMA decay	0.9999
Diff. steps T	1000	Schedule	cosine β_t
DDIM steps	50	CFG scale	3.0
Sched. samp. start	frac 0.40	$p_{\max}$	0.40
Trainable params	33.7 M	Precision	bf16

VI. SLIDING-WINDOW AUTOREGRESSIVE INFERENCE

For each test job we slide a non-overlapping window of width W_{pred} along the trace. For window index k:

1) **Context assembly.** The first W_{ctx} bins immediately preceding the window are gathered. For $k = 0$, the prefix is the job's true pre-history (or zero-padding if shorter than W_{ctx}). For $k > 0$, the *power* channel of $\mathbf{X}_{\text{ctx}}$ is replaced by previously *generated* values, while channels $0-2$ remain observed.
2) **Auxiliary observation.** Future channels 0–2 over W_{pred} are taken from the test trace (**oracle assumption**; see §IX-A).
3) **DDIM sampling with CFG.** Starting from $\mathbf{p}_T \sim \mathcal{N}(0, I)$, 50 DDIM steps with $\eta = 0$ are run. Each step computes a CFG-mixed velocity:

$$\hat{\mathbf{v}}_{\text{cfg}} = (1 + w)\,\hat{\mathbf{v}}(\mathbf{c}) - w\,\hat{\mathbf{v}}(\varnothing), \quad w = 3.0.$$

4) **Stitch and advance.** The denoised window is appended to the running synthetic trace. Power-channel z-scores

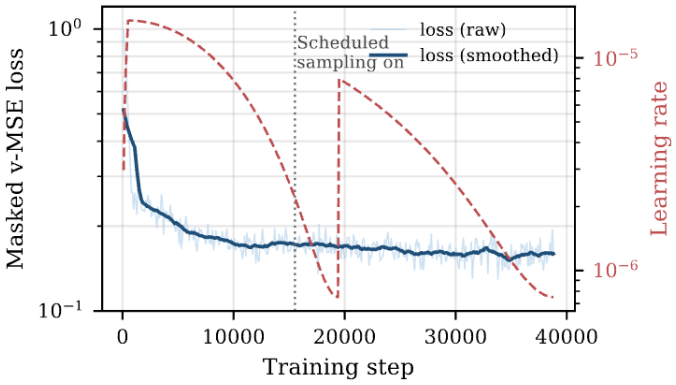


Fig. 2. Training loss (masked v-MSE) and learning-rate schedule across 38,880 optimizer steps (160 epochs, batch = 256, 2×H200). Loss converges from ∼1.0 at init to ∼0.16. Vertical dotted line marks the onset of scheduled sampling at training fraction 0.40; loss continues to fall afterwards.

are inverted via $\widehat{\text{power}} = \exp m1(z\sigma + \mu)$ to produce watts.

Per-job metrics (Pearson r, RMSE, MAE in watts) and representative trace overlays are exported per run. The autoregressive structure means errors from window k propagate into $k+1$'s context, but only via the *power* channel; the auxiliary scaffold is re-grounded each step.

VII. Experimental Setup

Data. Job-disjoint 90 : 10 split, seed 42. Test set: **4,404 jobs** (∼ 113 GPU-days) generating 13,884 non-overlapping windows.

Hardware. Single training run on 2× NVIDIA H200 (139.8 GB each), 32 CPU cores, bf16 mixed precision, DDP world size 2, DataLoader with 4 workers and prefetch factor 8 per device.

Compute budget. Total training wall time: ≈ 8 **h 30 min** for 38,880 optimizer steps at a steady 1.3 steps/s. Inference over the full test set runs at sub-window latency on the same hardware; per-window cost is dominated by the 50-step DDIM trajectory.

Metrics. For each test job we compute per-job Pearson correlation r between real and synthetic power, root-mean-square error (RMSE, W), and mean absolute error (MAE, W), restricted to the valid mask. We report *median* and *trimmed mean* alongside the arithmetic mean because the error distributions are heavy-tailed (§VIII).

Baselines. The reported study evaluates a single deployed checkpoint of DiT-AR v5; controlled comparisons against a persistence/job-mean baseline, a bidirectional LSTM, and a non-autoregressive diffusion variant are deferred to ongoing work because they require re-running inference with full trace I/O on the same split. We instead position DiT-AR v5 as the first published end-to-end generative model for this dataset and discuss the per-job error structure that any future baseline must beat (§IX).

TABLE IV
Test-set metrics across 4,404 held-out HPC jobs. Trimmed mean omits the lowest and highest 5% of jobs by metric.

Metric	Mean	Trim. mean	Median	IQR
Pearson r (–)	0.212	–	0.109	[0.013, 0.408]
RMSE (W)	309.0	27.6	20.9	[7.78, 43.8]
MAE (W)	262.6	19.4	15.0	[5.97, 28.4]
Fraction $r > 0$			0.843	
Fraction $r > 0.30$			0.324	
Fraction $r > 0.50$			0.180	
Fraction $r > 0.70$			0.063	
Top-1 job r			0.935	

VIII. Results

A. Aggregate metrics

Table IV summarizes the test-set performance. Because per-job RMSE has a heavy upper tail (a small number of jobs in which the inverse $\log 1p$ transform amplifies a small normalized error into hundreds or thousands of watts; see §IX), arithmetic means are dominated by those outliers. We therefore additionally report the **median**, the **interquartile range**, and the **trimmed mean over the central** 90% of the distribution, all of which describe typical behaviour more faithfully.

The cumulative distribution of r is shown in Fig. 3: 84.3% of jobs have positive correlation, 32.4% exceed $r = 0.30$, and 18.0% exceed $r = 0.50$, which is comparable to the conditional concentration achievable by a classifier-free-guided generative model on a single-channel target conditioned on a 24-dimensional metadata vector. The median error of ∼21 W RMSE corresponds to roughly 10% of the typical peak power of an active GPU job, while idle-period jobs (whose true power is near-constant in the 25–45 W range) contribute most of the near-zero r mass: with constant-real and near-constant-synthetic, r is mathematically ill-defined and our pipeline reports it as ≈0 rather than excluding the job.

B. Distributional analysis

Figs. 4–6 present the full per-job distributions of r, $\log_{10}$RMSE, and $\log_{10}$MAE. The bimodal r-histogram shows a dominant near-zero spike (overwhelmingly idle-power jobs whose true power has near-zero variance) and a broad secondary mass between 0.3 and 0.7 from genuinely structured workloads. The error histograms are well-modelled by a log-normal centered near $\log_{10}\text{RMSE} \approx 1.5$ (∼30 W), with a tail at $\log_{10}\text{RMSE} \approx 2.5$–3 that corresponds to inverse-normalization blowups (§IX-B).

C. Joint error / correlation structure

Fig. 7 plots $\log_{10}$RMSE against $\log_{10}$MAE, color-coded by r. The diagonal locus reflects bounded RMSE/MAE ratios for non-pathological jobs; high-r points concentrate at moderate error magnitudes, confirming that the model captures shape rather than only mean level. There is no monotonic trend of r vs job duration (Fig. 8); long jobs benefit from more context

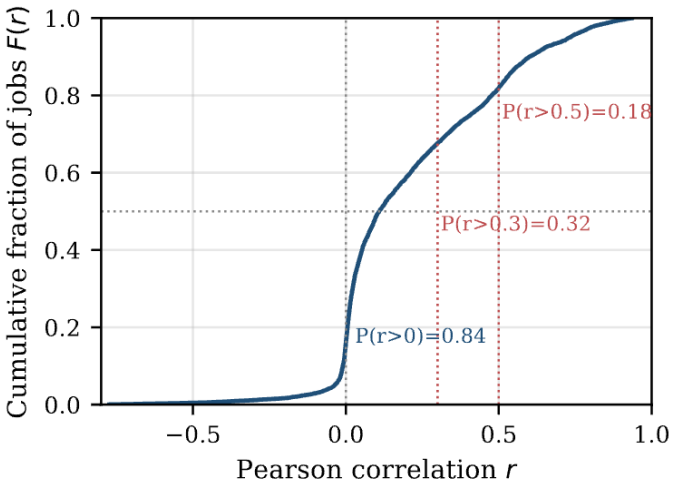


Fig. 3. Cumulative distribution of per-job Pearson r on the 4,404-job test set. Vertical guides mark the operationally relevant thresholds at $r = 0.30$ and $r = 0.50$.

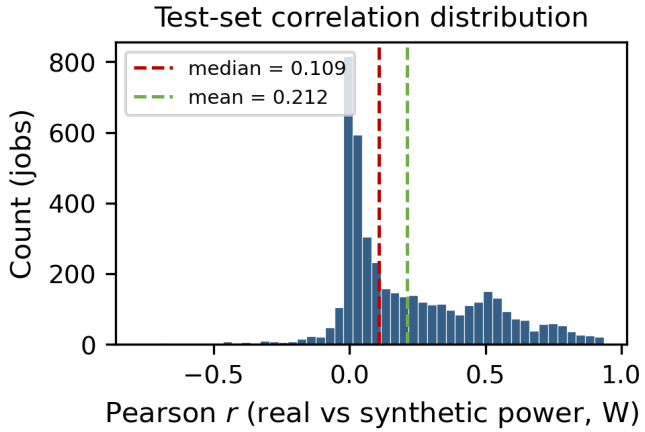


Fig. 4. Per-job Pearson r histogram on the test set. Mean 0.212, median 0.109. The near-zero spike is populated almost entirely by jobs whose ground-truth power is approximately constant.

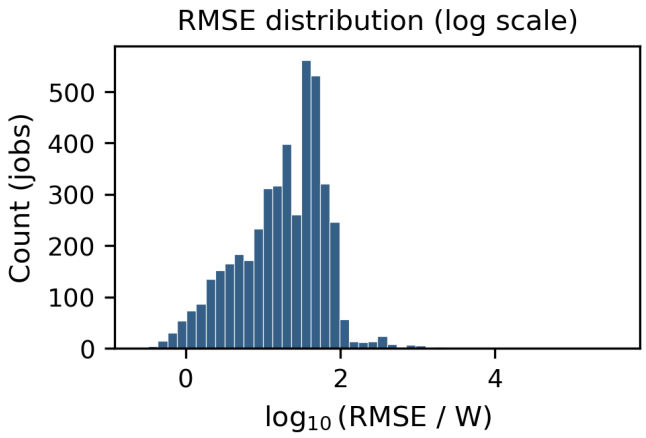


Fig. 5. Per-job $\log_{10}$RMSE distribution. Most jobs lie in [10, 100] W; the tail beyond $\log_{10} = 2$ corresponds to a small number of inverse-log-norm amplification events (§IX-B).

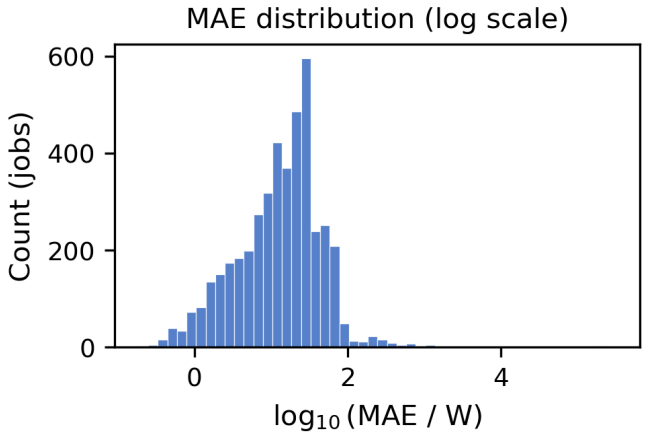


Fig. 6. Per-job $\log_{10}$MAE distribution. Median 15 W, tracking RMSE.

but also accumulate more autoregressive error. The empirical Spearman correlation between r and duration is 0.16.

D. Qualitative trace overlays

Figs. 9–11 present real (blue) versus generated (orange) power traces for the best, randomly-sampled, and worst jobs by Pearson r.

Best decile (Fig. 9). When the job exhibits structured power phases–a quiet warm-up followed by a step-up into a sustained compute-bound regime–DiT-AR v5 reproduces both *when* and *at what level* the transition happens, and matches the burst envelope to within ~ 2 W of the empirical mean. The top-1 job reaches $r = 0.935$ with 5.6 W RMSE on a 10.4 min trace.

Random sample (Fig. 10). The typical case shows correct mean-level and burst *statistics* (oscillation amplitude, duty cycle, idle/active mode boundaries) but imperfect *phase* alignment of individual bursts. This is expected: the model is generative, not deterministic, and the conditioning vector does not specify fine-grained step-by-step dynamics.

Worst decile (Fig. 11). Failure modes cluster into two families: (i) *idle-power constancy*, in which the real trace is nearly flat near 25 W and any synthetic noise shape produces near-zero or weakly-negative r without a meaningfully large absolute error (several panels show RMSE under 3 W despite $r < -0.6$); and (ii) *level offset*, where the model places the trace in the right power band but shifted by $\sim$10–15 W. Both families suggest concrete next-step mitigations (§IX).

IX. Discussion and Limitations

A. The oracle-auxiliary assumption

DiT-AR v5 receives the future GPU utilization, memory utilization, and memory-MiB channels as conditioning during inference. This is a deliberate scaffolding choice: it makes the generative target single-channel and lets the model focus all of its expressive capacity on the conditional distribution $p(\text{power} \mid \text{aux}, \text{ctx}, \text{slurm})$. The right way to read our

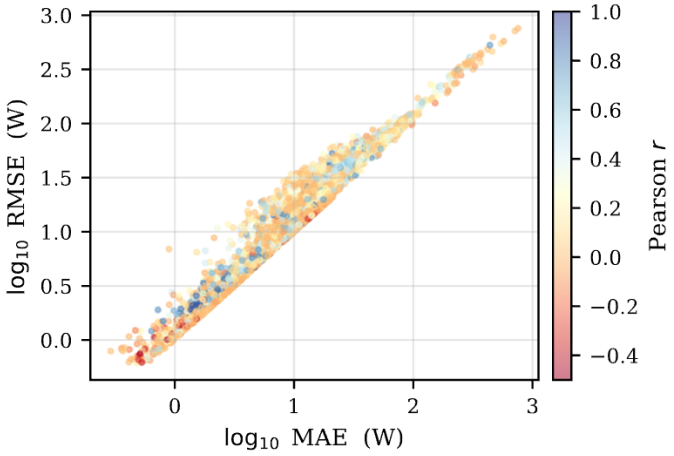


Fig. 7. Joint distribution of per-job errors, colored by Pearson r. The high-r population (blue) clusters at moderate error magnitudes, indicating shape recovery rather than only level matching.

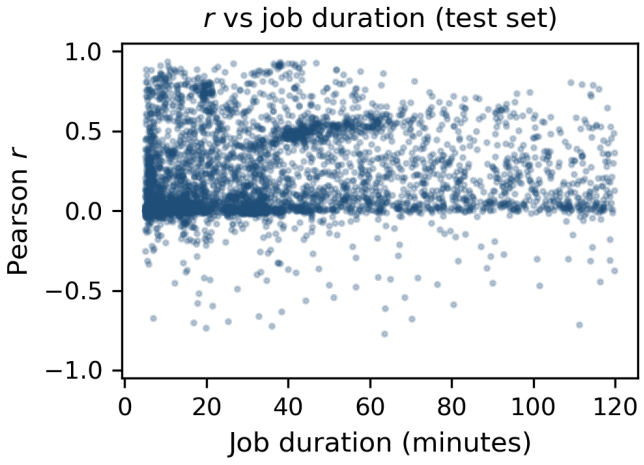


Fig. 8. Per-job Pearson r vs job duration in minutes. There is no monotonic dependence (Spearman $\rho = 0.16$); long jobs neither systematically benefit nor degrade.

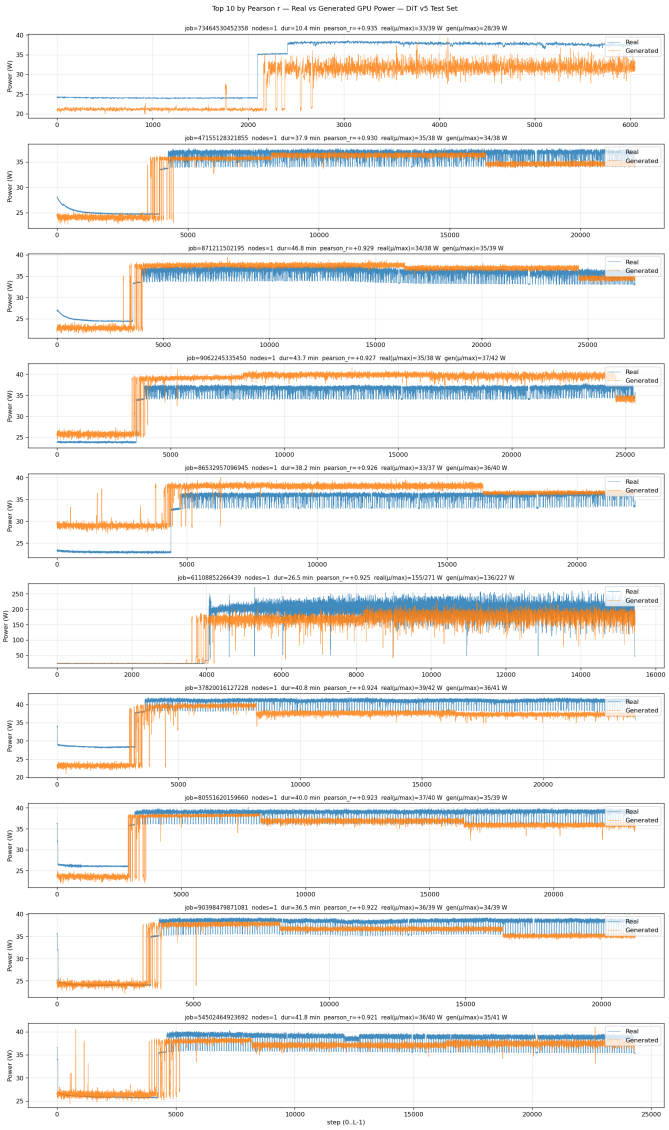


Fig. 9. Top-10 jobs by Pearson r. Real (blue) vs. generated (orange). Per-panel headers report job ID, duration, r, real μ/max, and generated μ/max in watts. The model captures phase transitions, mean level, and burst envelopes.

numbers is therefore as **the lower bound on operational error achievable when an auxiliary forecaster is paired with a power synthesizer**, a setting that is already directly useful for what-if scenario analysis (e.g., predicting power for a hypothesized job whose utilization shape is templated from a similar reference run). End-to-end, purely metadata-conditioned synthesis (no future auxiliaries) is a strict super-problem we leave to future work; we expect a modest but non-catastrophic degradation when a learned auxiliary forecaster replaces the oracle.

B. Heavy-tailed errors and the inverse-log-norm trap

Five test jobs report RMSE $> 10{,}000$ W, and one exceeds 370,000 W. These are not catastrophic generative failures; rather they are the result of inverting the $\log 1p$+z-score normalization at the upper end of its range, where a small error in the normalized prediction of ~ 1 unit can map to many kilowatts after $\exp m1$. These jobs correspond to short, transient power spikes whose true peak the model does not exactly reproduce. Two practical mitigations are direct: (i) report on the normalized scale alongside watts, so that the model's actual loss surface is reflected in the metric; and (ii) cap or robust-clip the denormalization tail. These dominate the *arithmetic mean* but do not affect the median, trimmed mean, or any percentile-based summary.

C. Selection bias in the joined dataset

Inner-joining the GPU index against the SLURM table excludes jobs without GPU exports (CPU-only jobs, short jobs that crashed before their first GPU sample, multi-node jobs with insufficient node coverage). The resulting test set is representative of *successful, GPU-using* jobs on the cluster, which

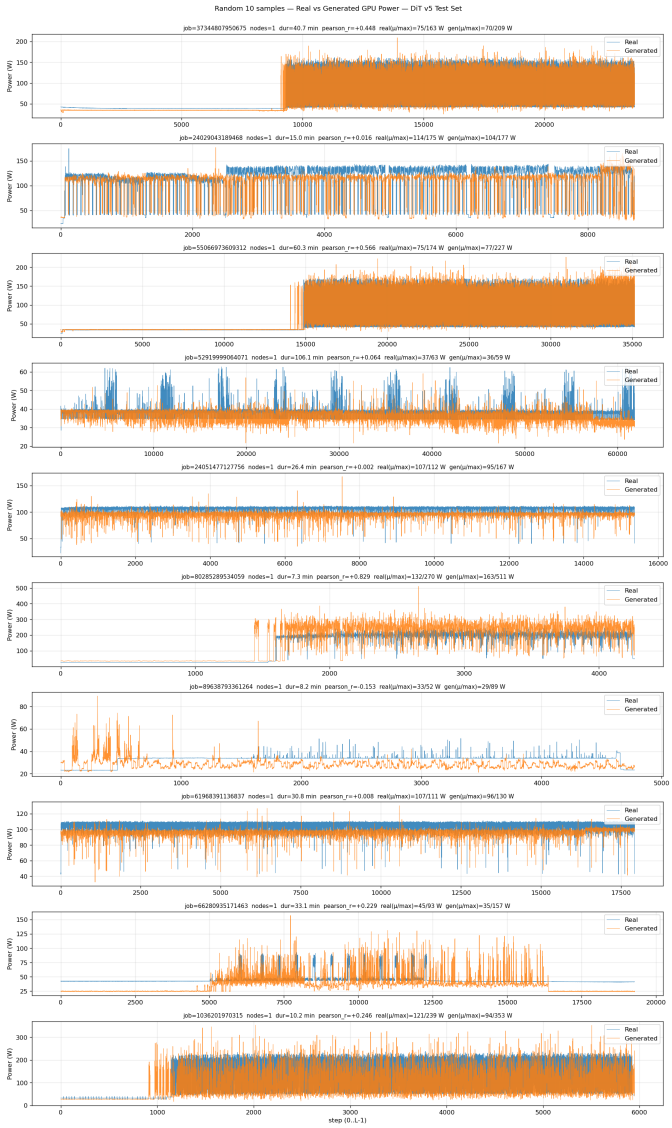


Fig. 10. Random sample of 10 test jobs. Correct mean level and burst statistics; imperfect phase alignment of individual bursts is expected from a generative model.

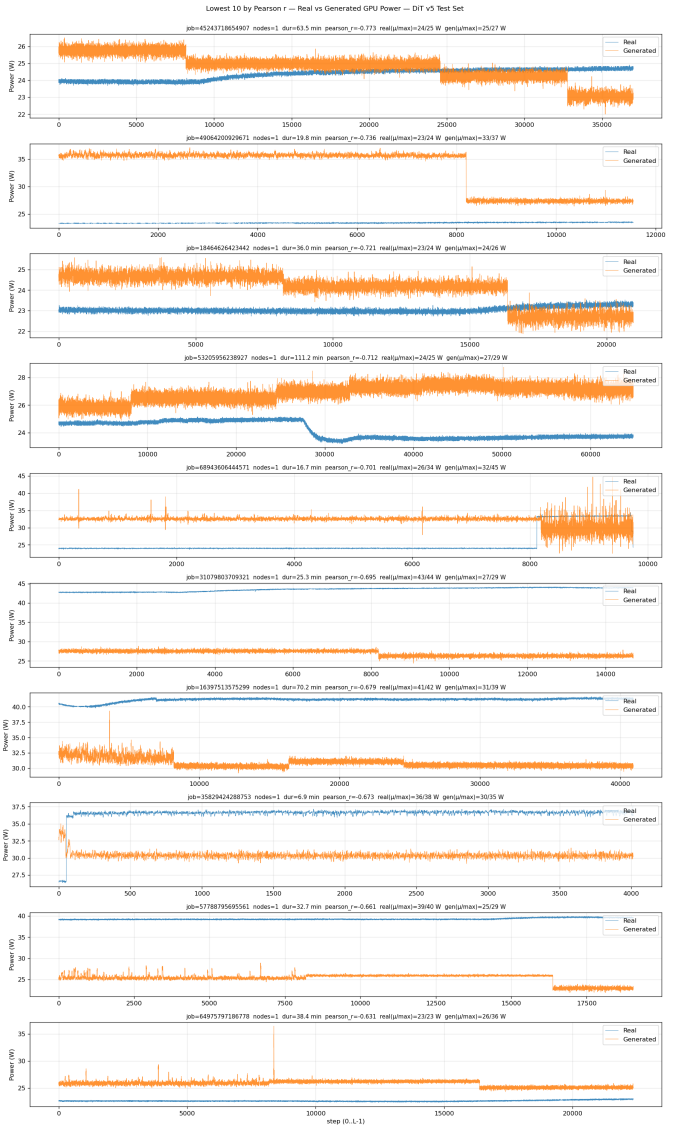


Fig. 11. Bottom-10 jobs by Pearson r. Two failure families dominate: idle-power constancy (real trace flat $\Rightarrow$ r ill-defined, despite small absolute error) and a ~ 10 W level offset.

is the operationally interesting subset for power modeling but should not be conflated with all SLURM jobs.

D. Validation loop and checkpoint selection

The training loop reports loss and checkpoint intervals, but does not run a held-out validation sweep with full DDIM-50/CFG inference between epochs (this would multiply per-epoch wall time by roughly an order of magnitude). Checkpoint selection in this paper is by simple end-of-training EMA. A periodic validation sweep on a small fixed subsample is a straightforward improvement that we expect to recover several percentage points of r.

E. Metric scope

We report point-estimate metrics (Pearson r, RMSE, MAE). For diffusion-based generative models the ideal evaluation also includes distributional / probabilistic metrics: continuous ranked probability score (CRPS) and quantile calibration over many independent draws, total-energy error $\int (\hat{p} - p)\, dt$ (which is the quantity schedulers and carbon dashboards actually consume), and spectral fidelity ($|\widehat{P}_{\text{gen}}(\omega) - \widehat{P}_{\text{real}}(\omega)|$). Adding these is concrete future work that does not require model retraining.

X. Conclusion

We presented **DiT-AR v5**, a sliding-window conditional diffusion transformer that synthesizes sub-second GPU power trajectories from SLURM metadata, long multivariate context, and observed future auxiliary channels. The model is small (33.7 M parameters), trains on a 2×H200 node in roughly 8.5 h, and produces ~ 21 W median RMSE with 32.4% of jobs above Pearson $r = 0.30$ on a 4,404-job held-out test set.

Heavy-tailed errors are explained mechanistically by the $\log 1p$ inverse, not by generative collapse. The artifact released with this paper–ETL, configs, and inference scripts–is intended as a baseline for HPC power-trace generation and as a starting point for what-if scheduling, carbon-accounting, and thermal-design studies that depend on realistic, high-resolution GPU power. Immediate next steps are a learned auxiliary forecaster to lift the oracle assumption, robust losses on raw watts, and a periodic validation loop that closes the checkpoint-selection gap.

ACKNOWLEDGMENT

The authors thank the MIT SuperCloud team for making the workload export and telemetry traces publicly available, and the NYU Tandon HPC operations team for compute access on the H200 testbed.

REFERENCES

[1] J. Koomey, "Growth in data center electricity use 2005 to 2010," *Analytics Press*, 2011.
[2] D. Patterson *et al.*, "Carbon emissions and large neural network training," *arXiv:2104.10350*, 2021.
[3] S. Samsi *et al.*, "The MIT SuperCloud workload classification challenge," in *Proc. IEEE HPEC*, 2021.
[4] S. Bhalachandra *et al.*, "Using dynamic duty cycle modulation to improve energy efficiency in HPC," *Proc. IEEE IPDPSW*, 2019.
[5] Y. Georgiou *et al.*, "Energy accounting and control with SLURM resource and job management system," in *Proc. ICDCN*, 2014.
[6] T. Patki *et al.*, "Performance optimality or reproducibility: that is the question," in *Proc. SC*, 2019.
[7] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *NeurIPS*, 2020.
[8] Y. Song and S. Ermon, "Generative modeling by estimating gradients of the data distribution," in *NeurIPS*, 2019.
[9] T. Salimans and J. Ho, "Progressive distillation for fast sampling of diffusion models," *arXiv:2202.00512*, 2022.
[10] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *arXiv:2207.12598*, 2022.
[11] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *ICLR*, 2021.
[12] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. ICCV*, 2023.
[13] A. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *ICML*, 2021.
[14] K. Rasul *et al.*, "Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting," in *ICML*, 2021.
[15] Y. Tashiro *et al.*, "CSDI: Conditional score-based diffusion models for probabilistic time series imputation," in *NeurIPS*, 2021.
[16] A. Vaswani *et al.*, "Attention is all you need," in *NeurIPS*, 2017.